

| GUÍA 5: Aplicando soluciones con Kotlin

Sigla	Asignatura	Experiencia de Aprendizaje
DSY1105	Desarrollo de Aplicaciones Móviles	EA1: Aplicando lenguajes de programacion para aplicaciones moviles
Tiempo	Modalidad de Trabajo	Indicadores de logro
1 hora	Parejas	IL1.2 IL 1.3



Antecedentes generales

Esta actividad busca que apliques conceptos avanzados de Kotlin, con un enfoque especial en la programación asíncrona mediante corrutinas y el modelado de estados con clases selladas (sealed classes) y clases de datos (data classes). Su propósito es que construyas un programa que maneje una operación de larga duración sin bloquear la ejecución, representando los posibles resultados de forma segura y expresiva.



Requerimientos para esta actividad

Para el desarrollo de esta actividad deberás disponer de:

- Conexión a internet
- IntelliJ IDEA
- JDK 17 o superior



Actividad

Deberás crear un programa que simule la **autenticación de un usuario** contra un servidor.

Este proceso es asíncrono y puede tener diferentes resultados. Implementarás una función que simule esta verificación usando corrutinas y modelarás los posibles estados del resultado (Autenticando, Éxito, Error) con una clase sellada.

Modelado de Datos y Estados

1. **Crear la Clase de Datos:**
 - Define una data class PerfilUsuario que contenga las propiedades del usuario que se obtendrían tras un login exitoso: nombreUsuario (String) y email (String). Las clases de datos son ideales para almacenar este tipo de información.



2. Crear la Jerarquía de Estados:

- Define una sealed class llamada ResultadoLogin. Las clases selladas son perfectas para representar un conjunto fijo y limitado de tipos, como los estados de una operación.
- Dentro de ResultadoLogin, define los tres posibles estados:
 - Una data class Exitó(val perfil: PerfilUsuario) que herede de ResultadoLogin.
 - Una data class Error(val mensaje: String) que herede de ResultadoLogin.
 - Un object Autenticando que herede de ResultadoLogin.

Creación de la Función Asíncrona

1. Crea una función suspend fun autenticarUsuario(usuario: String, contraseña: String): ResultadoLogin.
 - Las funciones suspend pueden ser pausadas y reanudadas, siendo el corazón de las corutinas.
2. Dentro de la función, simula una demora de red usando delay(2000L). Esto pausará la corutina por 2 segundos sin bloquear el hilo.
3. Añade una lógica simple de validación: si el usuario es "admin" y la contraseña es "1234", devuelve una instancia de ResultadoLogin.Exito con datos de un perfil de usuario de ejemplo.
De lo contrario, devuelve ResultadoLogin.Error con el mensaje "Credenciales incorrectas".

Ejecución y Manejo de Resultados

1. En tu función fun main(), utiliza runBlocking para crear un CoroutineScope que te permita llamar a tu función suspend.
2. Dentro de runBlocking, implementa la lógica para probar tu función:
 - Imprime en consola un mensaje indicando que el proceso está en estado Autenticando.
 - Llama a autenticarUsuario("admin", "1234") para iniciar la tarea asíncrona.
 - Utiliza una expresión when para manejar el ResultadoLogin devuelto. Debe haber un caso para Exito, Error y Autenticando.
 - Imprime un mensaje diferente y descriptivo en cada caso.
Por ejemplo, si el resultado es Exito, imprime el nombre de usuario y el email del perfil.
3. Repite el paso anterior, pero ahora llamando a la función con credenciales incorrectas (ej. "usuario", "0000") para probar el caso de Error.

Aplicando Funciones de Ámbito

- Modifica el caso Exito de tu expresión when para utilizar la función de ámbito let. Esto te permitirá ejecutar operaciones sobre el objeto perfil de una manera más segura y concisa.